Nombre: Cepeda Macías Raúl Emilio García Najera Eathan Omar Contreras Elizondo Luis Adrián	**Matrícula**: 2145193 2144814 2145018
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 5	**Actividad**: Actividad fundamental 5 **Link de Github:** https://github.com/rulocm07/Unidad-5-CLN
Fecha: 05/11/2026	

Bibliografia (Formato APA)

pytest-dev. (s. f.). Get started — pytest documentation. https://docs.pytest.org/en/stable/getting-started.html

SQLAlchemy. (s. f.). Working with transactions and the DBAPI. https://docs.sqlalchemy.org/en/20/orm/session_transaction.html

Python Software Foundation. (s. f.). unittest.mock — Mock object library. https://docs.python.org/3/library/unittest.mock.html

Gómez Marroquín, M. A. (2026). Laboratorio guiado: Unidad 5, integración, pruebas y calidad [Material de clase]. FIME, UANL.

Fowler, M. (2007). Mocks aren't stubs. https://martinfowler.com/articles/mocksArentStubs.html

Objetivo general

Implementar una estrategia integral de pruebas automatizadas y aseguramiento de calidad sobre la lógica de negocios de TechStore. La práctica contempla pruebas unitarias con pytest, el ciclo TDD Red–Green–Refactor, pruebas de integración con SQLAlchemy y SQLite, simulaciones de servicios externos mediante unittest.mock, validación de permisos, análisis estático con Ruff, medición de cobertura e integración continua mediante GitHub Actions.

Como resultado se documentan las nueve evidencias E1 a E9, la matriz de 12 casos del PIA y el comportamiento del pipeline ante una ejecución exitosa, un fallo controlado y la recuperación posterior.

1. Introducción y arquitectura de calidad

TechStore es un ejemplo de comercio electrónico cuyo dominio incluye pedidos, descuentos, control de inventario, pagos, persistencia y autorización. Estas reglas deben verificarse de forma reproducible para disminuir errores que pudieran afectar la consistencia de la operación.

La solución distingue cuatro responsabilidades: dominio, con las reglas de las entidades; aplicación, con la coordinación del servicio de cobro; persistencia, mediante SQLAlchemy y SQLite en memoria; y seguridad, con las reglas de cancelación. El conjunto de pruebas con pytest revisa las rutas relevantes y GitHub Actions ejecuta automáticamente controles de calidad en cada cambio registrado.

2. Evidencias de laboratorio (E1 a E9)

Evidencia E1 · Instalación y comprobación inicial

Se creó y activó un entorno virtual de Python para separar las bibliotecas utilizadas por el laboratorio. Después se instalaron las dependencias establecidas en requirements.txt y se ejecutó la suite inicial con pytest.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pytest -v
```

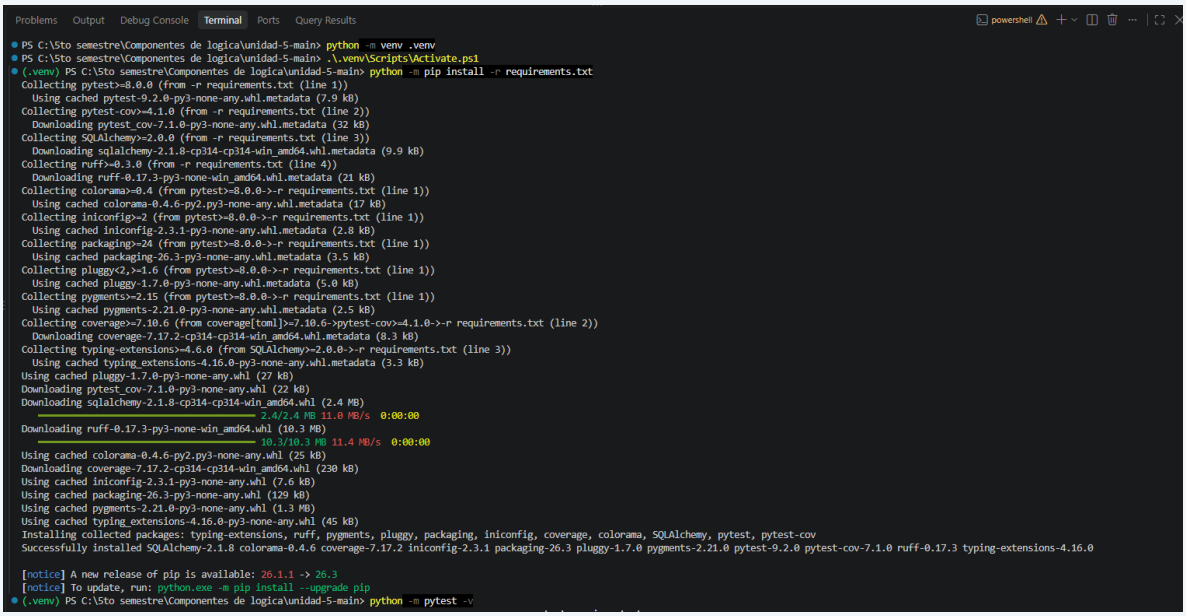


Figura E1.1. Creación y activación del entorno virtual e instalación de dependencias.

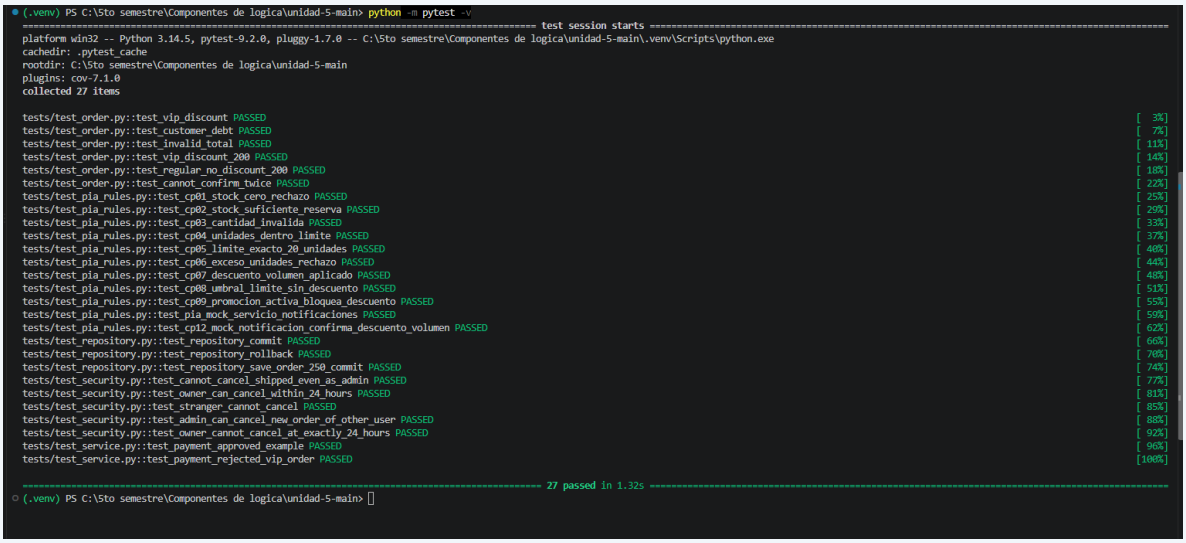


Figura E1.2. Primera ejecución de pytest: 27 pruebas aprobadas.

Resultado observado: 27 pruebas PASSED. El entorno virtual evita conflictos de dependencias con otros proyectos y pytest localiza y ejecuta las funciones de prueba, mostrando errores o confirmaciones de éxito en la terminal.

Evidencia E2 · Comprensión de pruebas unitarias y descuentos

Se comprobaron las reglas de confirmación de pedidos para clientes VIP y regulares. La condición VIP aplica un descuento de 15%; un pedido de $200 debe generar un cobro final de $170, mientras que el mismo importe para un cliente regular permanece en $200.

```
def test_vip_discount_200():
    order = Order(200, vip=True)
    assert order.confirm() == 170
    assert order.status == "CONFIRMED"

def test_regular_no_discount_200():
    order = Order(200, vip=False)
    assert order.confirm() == 200
    assert order.status == "CONFIRMED"
```

Entrada	Regla	Esperado	Resultado	Estado
Order(100, vip=True)	Descuento VIP del 15%	$85; CONFIRMED	$85; CONFIRMED	PASSED
Order(200, vip=True)	Descuento VIP del 15%	$170; CONFIRMED	$170; CONFIRMED	PASSED
Order(200, vip=False)	Sin descuento	$200; CONFIRMED	$200; CONFIRMED	PASSED
Order(100, has_debt=True)	Deuda pendiente	ValueError	ValueError	PASSED
Order(0) y Order(-50)	Importe no positivo	ValueError	ValueError	PASSED

```
python -m pytest tests/test_order.py -v
```

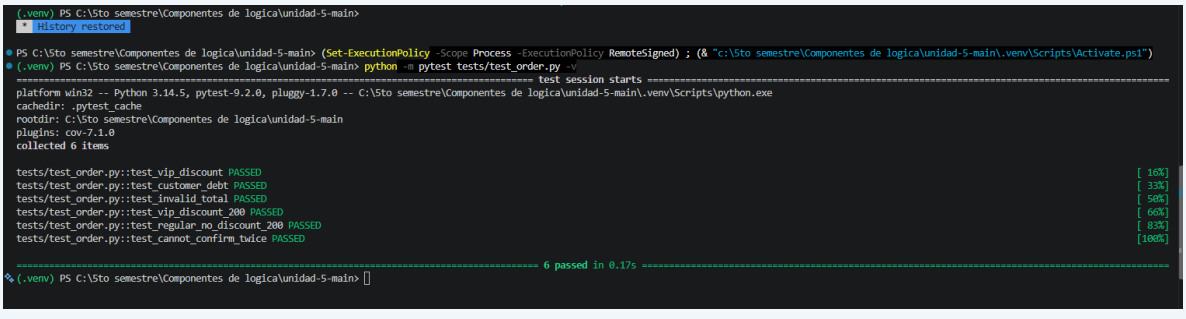


Figura E2.1. Ejecución de seis pruebas unitarias del dominio con resultado satisfactorio.

Las pruebas siguen el esquema Arrange–Act–Assert: preparan el pedido, ejecutan la operación confirm() y verifican el importe calculado y el estado final.

Evidencia E3 · Ciclo TDD real: Red → Green → Refactor

Se verificó una regla adicional: un pedido ya confirmado no puede confirmarse por segunda ocasión. Como la solución se encontraba implementada en el proyecto recibido, se desactivó temporalmente la validación para reproducir la fase RED y posteriormente se restauró, conservando la prueba original.

Fase RED · fallo esperado

```
def test_cannot_confirm_twice():
    order = Order(100)
    order.confirm()
    with pytest.raises(ValueError, match="already confirmed"):
        order.confirm()
```

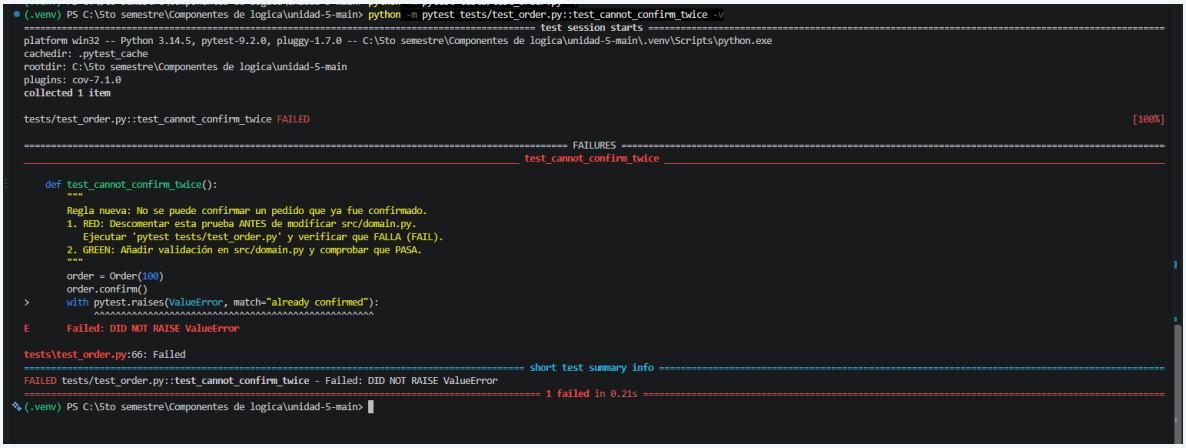


Figura E3.1. Fase RED: la prueba falla con DID NOT RAISE ValueError al permitir la doble confirmación.

Fase GREEN · validación restaurada

```
if self.status == "CONFIRMED":
    raise ValueError("already confirmed")
```

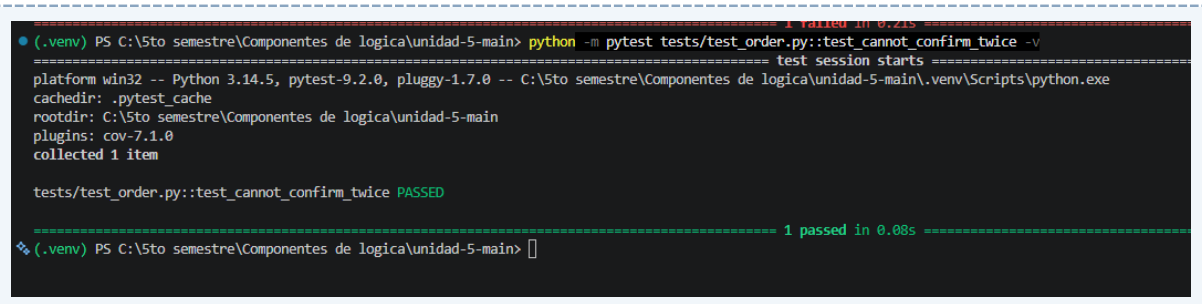


Figura E3.2. Fase GREEN: la misma prueba se ejecuta correctamente después de restaurar la validación.

Comprobación general y reflexión Refactor

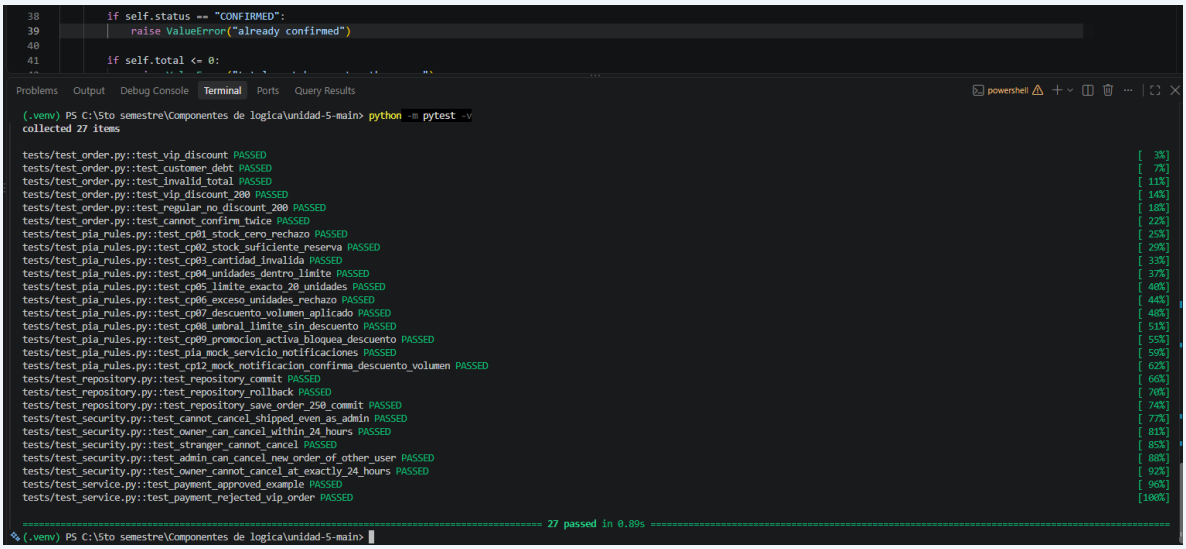


Figura E3.3. Código de confirmación restaurado y ejecución de la suite completa: 27 pruebas aprobadas.

Se comprobó que la condición es simple, mantiene la interfaz pública y no altera el comportamiento de las demás pruebas. TDD exige que primero exista una prueba capaz de fallar, después un cambio mínimo que la haga pasar y, por último, una revisión del diseño sin modificar el resultado. En esta reproducción no se atribuye un refactor adicional que no se haya ejecutado.

Evidencia E4 · Integración con base de datos y transacciones

Se realizaron pruebas de integración sobre un repositorio SQLAlchemy conectado a SQLite en memoria. Se verificaron las operaciones de guardado, recuperación de registros y reversión de una transacción.

```
def test_repository_save_order_250_commit(session):
    repo = SqlOrderRepository(session)
    order = OrderRow(id=2, total=250.0, status="CONFIRMED")
    repo.save(order)
    session.commit()

    saved = repo.get_by_id(2)
    assert saved is not None
    assert saved.id == 2
    assert saved.total == 250.0
    assert saved.status == "CONFIRMED"
```

```python
def test_repository_save_order_250_commit(session):
    """
    Actividad Paso 4: Guarda un pedido de total 250 con id=2,
    hace commit y verifica id, total y estado.
    """
    repo = SqlOrderRepository(session)
    order = OrderRow(id=2, total=250.0, status="CONFIRMED")
    repo.save(order)
    session.commit()

    saved = repo.get_by_id(2)
    assert saved is not None
    assert saved.id == 2
    assert saved.total == 250.0
    assert saved.status == "CONFIRMED"


```

Figura E4.1. Código que guarda un pedido de $250 con ID 2 y comprueba los datos persistidos.

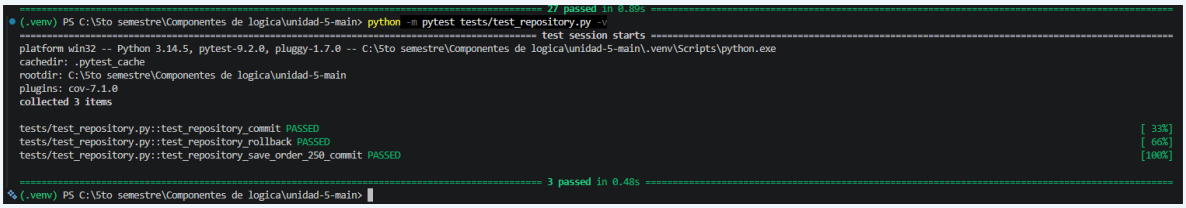


Figura E4.2. Ejecución de tres pruebas: commit, rollback y almacenamiento del pedido de $250.

Commit confirma las operaciones pendientes de una transacción; rollback revierte las modificaciones no confirmadas. Esta comprobación es de integración porque intervienen el modelo ORM, la sesión y el motor de base de datos; una prueba unitaria de dominio no cubre esas interacciones.

Evidencia E5 · Servicios externos sin cobros reales (mocks)

Se utilizó unittest.mock.Mock para simular la pasarela de cobro y comprobar tanto la aprobación como el rechazo de un pago sin ejecutar transferencias financieras reales. Para un pedido VIP de $200, el importe que debe recibir el mock es $170.

```python
def test_payment_rejected_vip_order():
    gateway = Mock()
    gateway.charge.return_value = False
    order = Order(200, vip=True)
    service = OrderService(gateway)
    with pytest.raises(RuntimeError, match="payment rejected"):
        service.checkout(order)
    gateway.charge.assert_called_once_with(170)
```

```python
def test_payment_rejected_vip_order():
    """
    Actividad Paso 5: Prueba de rechazo para un pedido VIP de $200.
    - Verifica que el intento de cobro fue por $170 (15% de descuento).
    - Verifica que se lanzó RuntimeError("payment rejected").
    """
    gateway = Mock()
    gateway.charge.return_value = False

    order = Order(200, vip=True)
    service = OrderService(gateway)

    with pytest.raises(RuntimeError, match="payment rejected"):
        service.checkout(order)

    gateway.charge.assert_called_once_with(170)
```

Figura E5.1. Código que simula el rechazo y verifica el intento de cobro por $170.

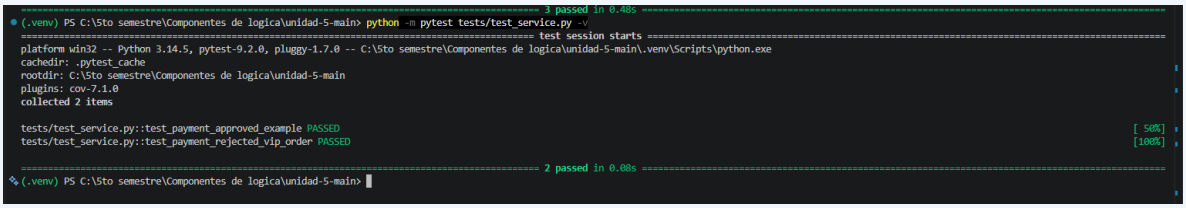


Figura E5.2. Prueba de pago aprobado y prueba de rechazo VIP con resultado PASSED.

Riesgo detectado: en la implementación base, el pedido puede cambiar a CONFIRMED antes de conocer el resultado del cobro. Un rechazo dejaría un estado inconsistente.

Propuesta de mejora: manejar un estado transitorio PENDING_PAYMENT y pasar a CONFIRMED únicamente cuando la pasarela confirme el pago; en caso contrario, registrar el rechazo. Esta solución se presenta como propuesta y no como una modificación ya aplicada.

Evidencia E6 · Seguridad y matriz de autorización

Se verificaron las restricciones de cancelación según rol, propiedad del pedido, estado y antigüedad. La regla de máxima prioridad impide cancelar cualquier pedido SHIPPED. Los administradores pueden cancelar pedidos no enviados, mientras que los propietarios no administradores tienen una ventana estrictamente menor de 24 horas.

```python
def test_admin_can_cancel_new_order_of_other_user():
    now = datetime(2026, 1, 1, 12, 0)
    created = now - timedelta(hours=48)
    assert can_cancel("NEW", owner="alice", user="admin",
                      is_admin=True, created_at=created, now=now)

def test_owner_cannot_cancel_at_exactly_24_hours():
    now = datetime(2026, 1, 2, 12, 0)
    created = datetime(2026, 1, 1, 12, 0)
    assert not can_cancel("NEW", owner="alice", user="alice",
                          is_admin=False, created_at=created, now=now)
```

```python
def test_admin_can_cancel_new_order_of_other_user():
    """Un administrador cancela un pedido NEW de otro usuario (debe permitirse)."""
    now = datetime(2026, 1, 1, 12, 0)
    created = now - timedelta(hours=48)  # Más de 24 horas
    assert can_cancel("NEW", owner="alice", user="admin", is_admin=True, created_at=created, now=now)


def test_owner_cannot_cancel_at_exactly_24_hours():
    """El propietario intenta cancelar exactamente a las 24 horas (debe rechazarse)."""
    now = datetime(2026, 1, 2, 12, 0)
    created = datetime(2026, 1, 1, 12, 0)  # Diferencia exacta de 24 horas
    assert not can_cancel("NEW", owner="alice", user="alice", is_admin=False, created_at=created, now=now)
```

Figura E6.1. Código de autorización para administrador y límite exacto de 24 horas.

Solicitante	¿Propietario?	Estado	Antigüedad	Resultado
Administrador	No	SHIPPED	1 hora	Denegado
Administrador	No	NEW	48 horas	Permitido
Propietario	Sí	CONFIRMED	2 horas	Permitido
Propietario	Sí	NEW	24 horas	Denegado
Propietario	Sí	NEW	25 horas	Denegado
Tercero regular	No	NEW	2 horas	Denegado

```
========== 2 passed in 0.08s ==========
(.venv) PS C:\5to semestre\Componentes de logica\unidad-5-main> python -m pytest tests/test_security.py -v
========== test session starts ==========
platform win32 -- Python 3.14.5, pytest-9.2.0, pluggy-1.7.0 -- C:\5to semestre\Componentes de logica\unidad-5-main\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\5to semestre\Componentes de logica\unidad-5-main
plugins: cov-7.1.0
collected 5 items

tests/test_security.py::test_cannot_cancel_shipped_even_as_admin PASSED                    [ 20%]
tests/test_security.py::test_owner_can_cancel_within_24_hours PASSED                       [ 40%]
tests/test_security.py::test_stranger_cannot_cancel PASSED                                 [ 60%]
tests/test_security.py::test_admin_can_cancel_new_order_of_other_user PASSED               [ 80%]
tests/test_security.py::test_owner_cannot_cancel_at_exactly_24_hours PASSED                [100%]

========== 5 passed in 0.08s ==========
(.venv) PS C:\5to semestre\Componentes de logica\unidad-5-main>
```

Figura E6.2. Ejecución de cinco pruebas de autorización, todas aprobadas.

El uso de fechas fijas y la inyección del parámetro now evita resultados dependientes del reloj del sistema, especialmente para la frontera exacta de 24 horas.

Evidencia E7 · Cobertura de código y análisis de calidad

El análisis estático se ejecutó con Ruff, seguido por la suite de pytest con medición de cobertura sobre los módulos de src/. El requisito del laboratorio es alcanzar por lo menos el 80% de líneas ejecutadas.

```
python -m ruff check src tests
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
```

```
(.venv) PS C:\5to semestre\Componentes de logica\unidad-5-main> python -m ruff check src tests
All checks passed!
(.venv) PS C:\5to semestre\Componentes de logica\unidad-5-main>
```

Figura E7.1. Ruff termina con el mensaje All checks passed!

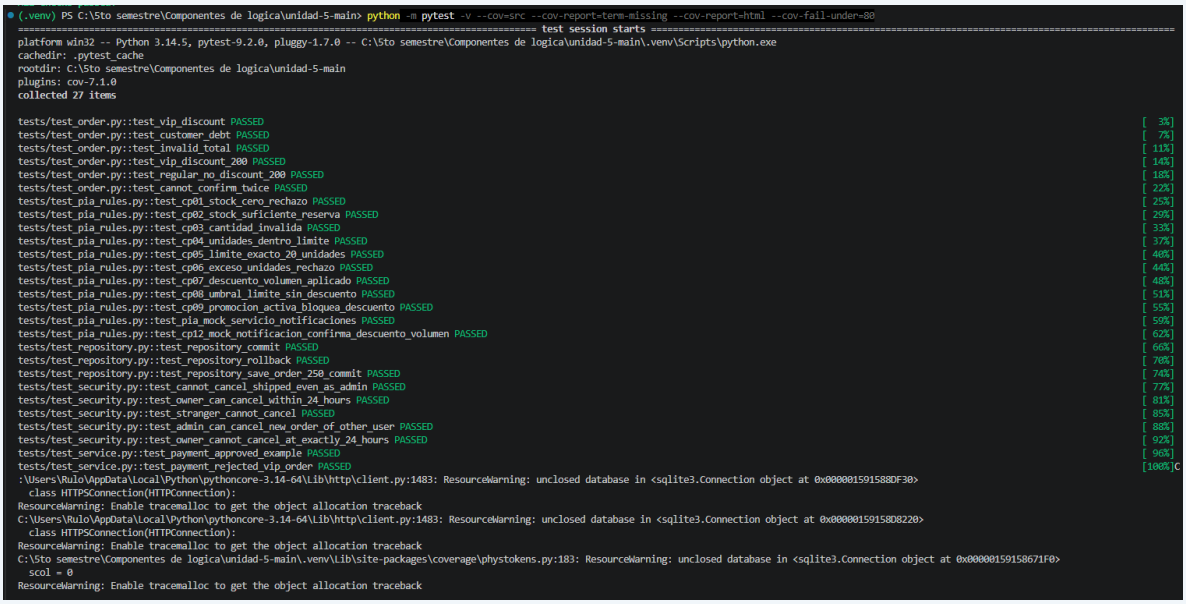


Figura E7.2. Ejecución de las 27 pruebas con pytest y mensajes informativos de la terminal.

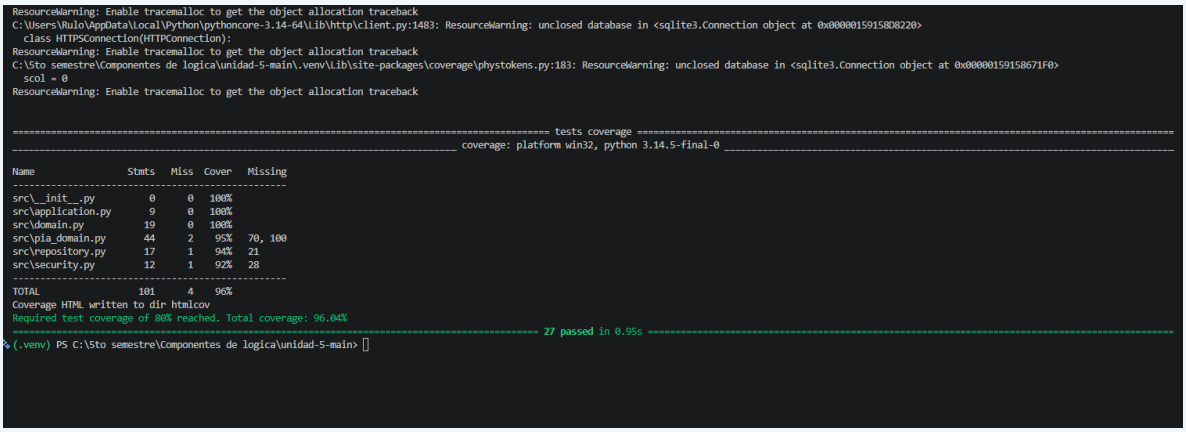


Figura E7.3. Resumen de cobertura y total calculado en terminal.

Módulo	Sentencias	No cubiertas	Cobertura
src/application.py	9	0	100%
src/domain.py	19	0	100%
src/pia_domain.py	44	2	95%
src/repository.py	17	1	94%
src/security.py	12	1	92%
TOTAL	101	4	96%

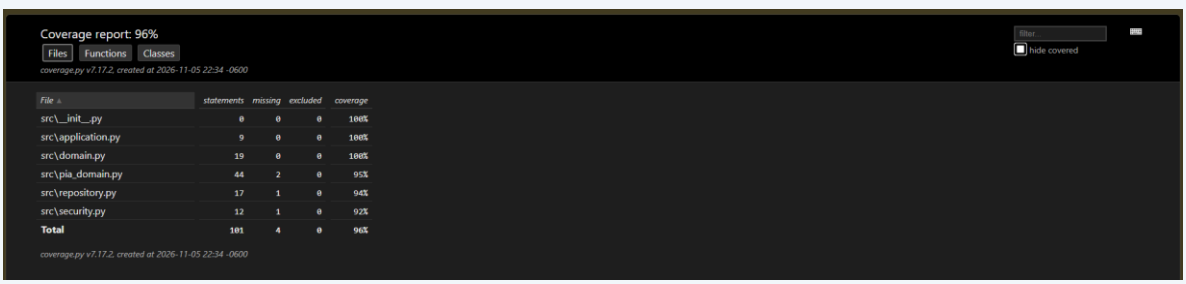


Figura E7.4. Reporte HTML de cobertura global, mostrado en el navegador.

La cobertura global de esta ejecución fue de 96.04% (redondeada al 96% en HTML), con 27 pruebas aprobadas. Los módulos sin cobertura completa fueron pia_domain.py, repository.py y security.py. Medir cobertura no equivale a demostrar ausencia de defectos: una prueba debe verificar resultados y límites relevantes.

La terminal emitió advertencias ResourceWarning por conexiones SQLite no cerradas completamente. No invalidaron las pruebas, pero señalan una oportunidad de mejora en la limpieza de recursos usados durante los tests.

Evidencia E8 · Integración continua con GitHub Actions

El proyecto se publicó en un repositorio propio y se comprobó el workflow CI Business Logic. Se registraron tres momentos: una ejecución inicial exitosa en main, una ejecución fallida en la rama ci/controlled-failure y una ejecución exitosa tras retirar la prueba temporal.

Repositorio: https://github.com/rulocm07/Unidad-5-CLN

Workflow: .github/workflows/ci.yml — CI Business Logic.

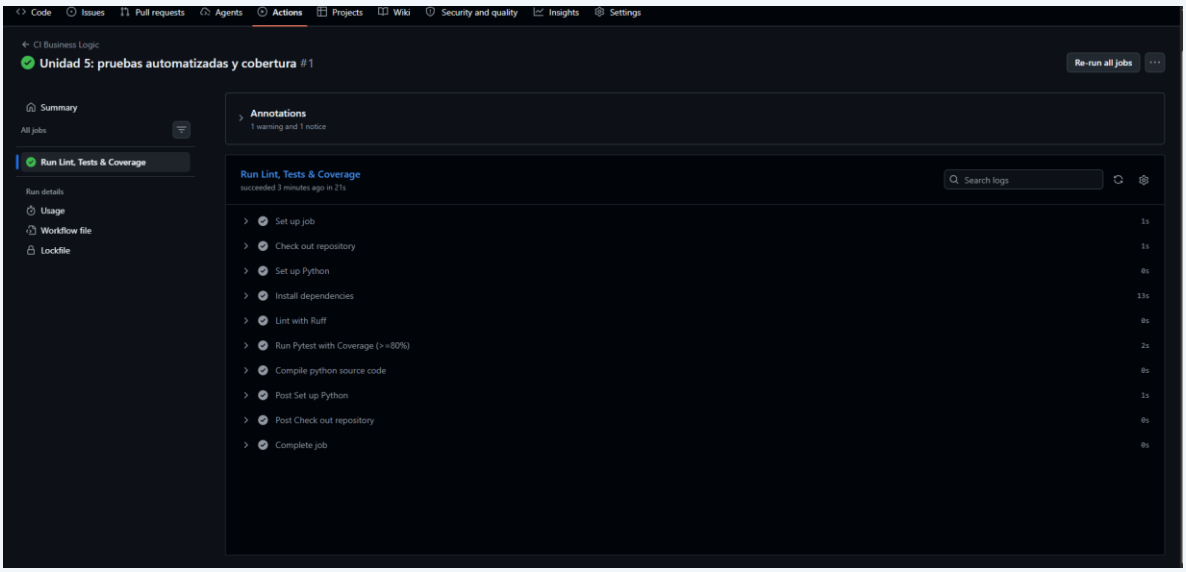


Figura E8.1. Primera ejecución exitosa de GitHub Actions: instalación, Ruff, pytest y compilación aprobados.

Para demostrar la detección de regresiones se introdujo una prueba temporal deliberadamente falsa. En un primer intento Ruff advirtió sobre una comparación de constantes; el caso se ajustó para que Ruff pasara y la comprobación fallida fuera detectada específicamente por pytest.

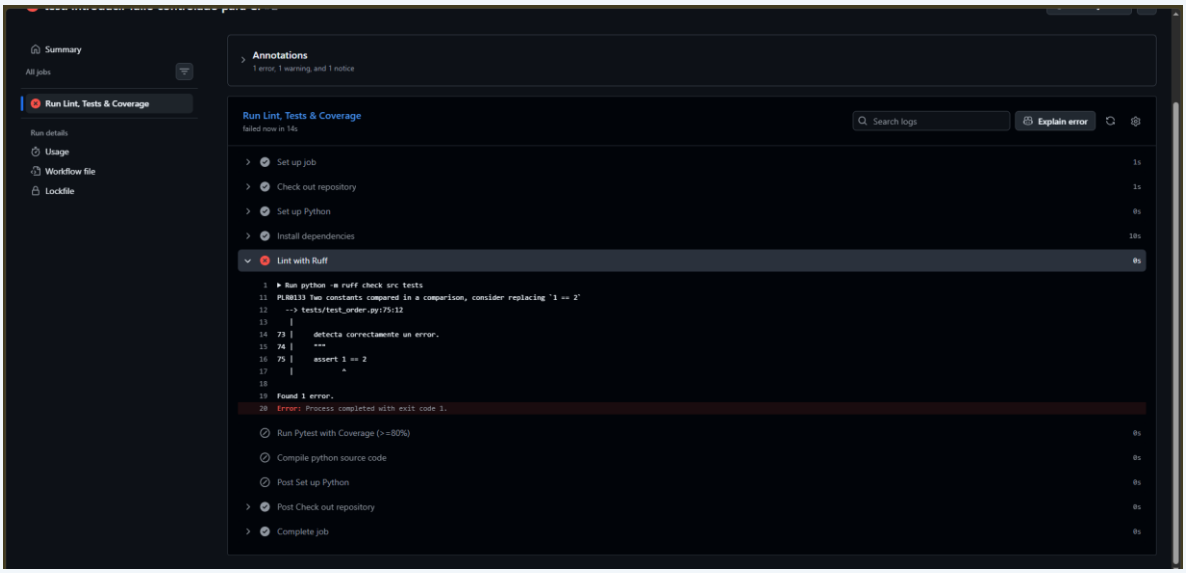


Figura E8.2. Ejecución RED: Ruff aprueba y pytest rechaza la prueba temporal.

En la ejecución fallida, pytest reportó 1 fallo y 27 pruebas aprobadas. Aunque la cobertura cumplía el umbral, el job quedó marcado como fallido por la aserción incorrecta. La rama principal main no fue modificada para provocar este error.

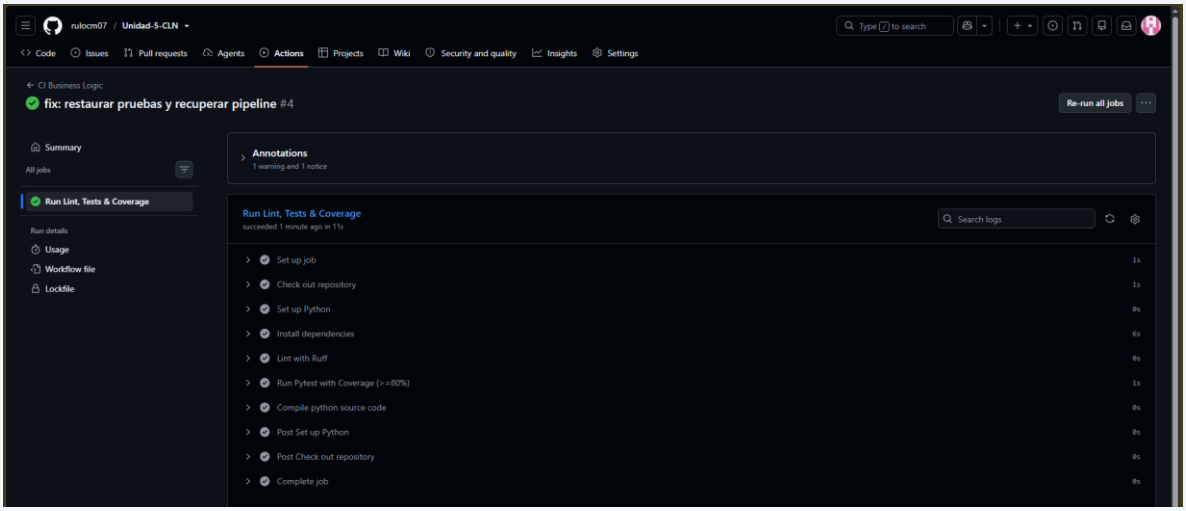


Figura E8.3. Ejecución GREEN de recuperación después de eliminar la prueba temporal.

La recuperación demuestra que el pipeline vuelve a un estado exitoso cuando se restablecen las pruebas correctas. Un workflow fallido evita que esa ejecución sea considerada satisfactoria; para bloquear formalmente fusiones en GitHub se requiere configurar reglas de protección de ramas.

Evidencia E9 · Aplicación de pruebas al PIA de TechStore

Se seleccionaron tres reglas: control de inventario, límite de 20 unidades por pedido y descuento por volumen del 5% para importes mayores a $5,000 sin promoción vigente. La matriz incorpora además persistencia, rechazo de pagos y notificación simulada, hasta sumar 12 casos identificados entre CP-01 y CP-12.

ID	Regla	Entrada	Resultado esperado	Tipo
CP-01	Stock	0 disponibles, solicita 1	Rechazo: inventario insuficiente	Límite
CP-02	Stock	5 disponibles, reserva 2	Stock restante: 3	Éxito
CP-03	Stock	Cantidad = −1	BusinessRuleViolation	Error
CP-04	Unidades	15 unidades	Pedido permitido	Éxito
CP-05	Unidades	20 unidades exactas	Pedido permitido	Límite
CP-06	Unidades	21 unidades	Rechazo por exceder 20	Error
CP-07	Descuento	Subtotal $6,000, sin promoción	5%: total $5,700	Éxito
CP-08	Descuento	Subtotal $5,000 exactos	Sin descuento	Límite
CP-09	Descuento	Subtotal $6,000, con promoción	Sin descuento por volumen	Error
CP-10	Persistencia	ID 2, total $250 y commit	Registro guardado en SQLite	Integración
CP-11	Pago mock	Pedido VIP $200 rechazado	Intento $170 y RuntimeError	Error/Mock
CP-12	Aviso mock	Subtotal $6,000 sin promoción	Confirma y notifica $5,700	Éxito/Mock

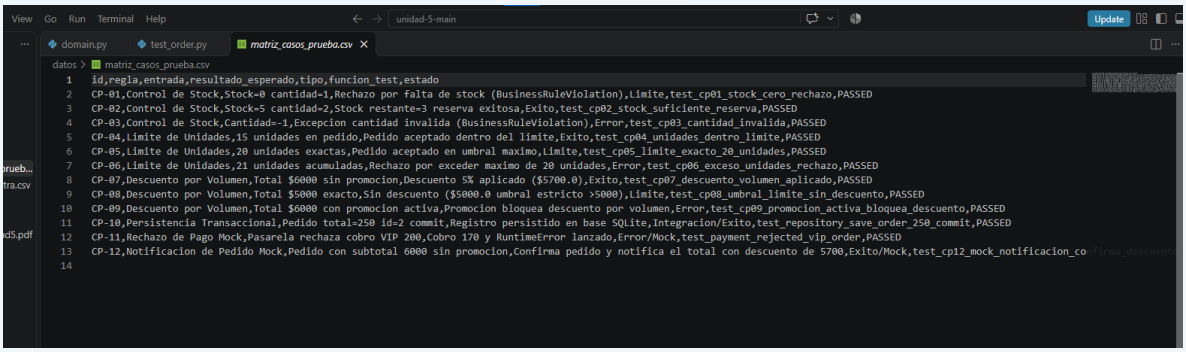


Figura E9.1. Matriz de pruebas en formato CSV, del caso CP-01 al CP-12.

El archivo tests/test_pia_rules.py reúne 11 pruebas: nueve para las tres reglas principales, una comprobación auxiliar de notificaciones y la prueba CP-12. El caso CP-10 se ejecuta en tests/test_repository.py y CP-11 en tests/test_service.py; ambos ya se verificaron también en E4 y E5.

```
python -m pytest tests/test_pia_rules.py -v
python -m pytest tests/test_repository.py::test_repository_save_order_250_commit -v
```

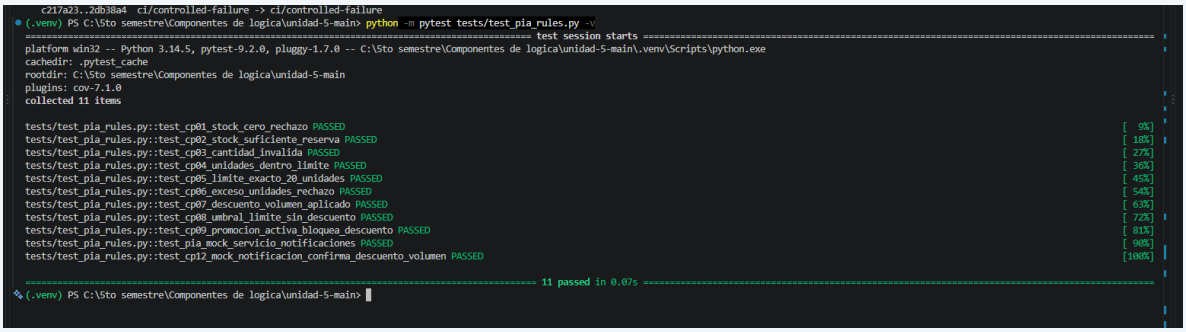


Figura E9.2. Ejecución de tests/test_pia_rules.py: 11 pruebas aprobadas.

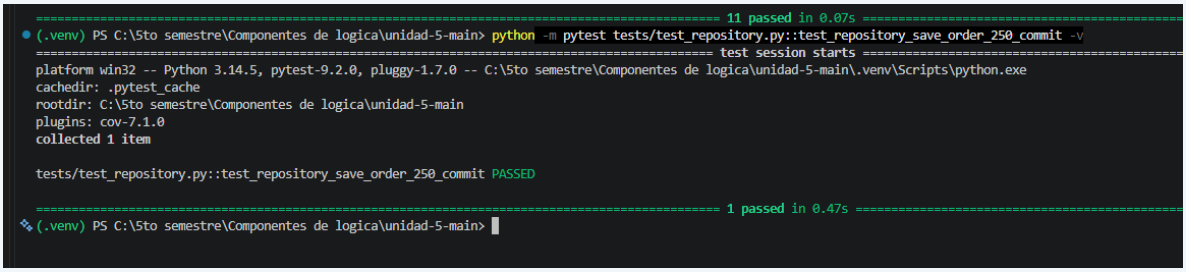


Figura E9.3. Prueba individual CP-10, de persistencia en SQLite: 1 prueba aprobada.

La medición de cobertura para todos los archivos src/ usando solo el subconjunto de pruebas del PIA fue de 57.43%, porque los módulos de aplicación, dominio general y seguridad no se ejecutaron. Para evaluar el módulo pertinente al PIA se realizó una medición específica sobre src.pia_domain.

```
python -m pytest tests/test_pia_rules.py --cov=src.pia_domain \
    --cov-report=term-missing --cov-report=html --cov-fail-under=80 -v
```

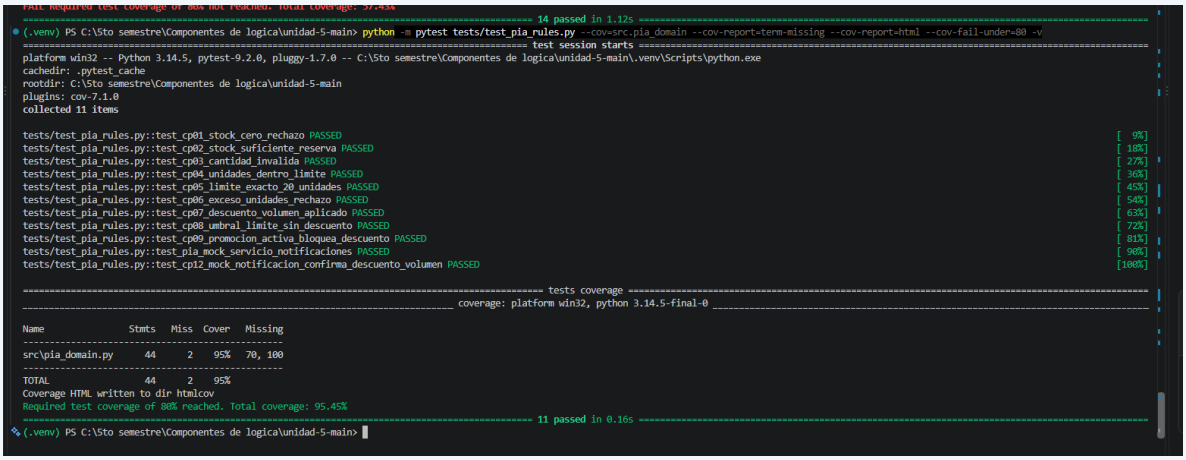


Figura E9.4. Cobertura específica del módulo del PIA: 11 passed, 95.45% y umbral de 80% cumplido.

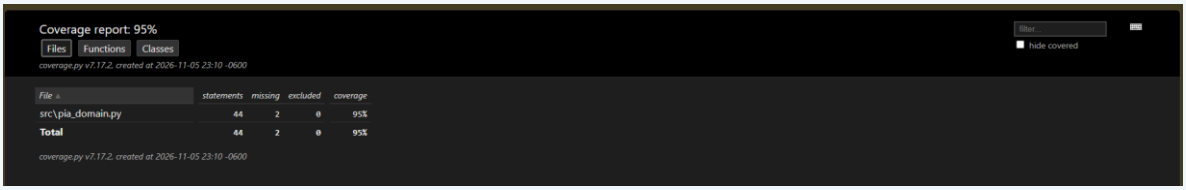


Figura E9.5. Reporte HTML de src/pia_domain.py: 44 sentencias, dos no cubiertas y 95% redondeado.

El módulo src/pia_domain.py alcanzó una cobertura exacta de 95.45% (42 de 44 sentencias), con dos líneas no ejecutadas: la validación defensiva de cantidad no positiva al agregar artículos y la confirmación de un pedido sin artículos. Se distinguen así los resultados de la cobertura del PIA y los de la suite global de E7.

2.1 Uso responsable y revisión de asistencia con IA

Durante la preparación del laboratorio se utilizó asistencia de IA para comprender los comandos y ordenar el material de evidencias. Las capturas y resultados de las pruebas fueron comprobados mediante ejecuciones locales y en GitHub Actions. Para cualquier prueba sugerida automáticamente, es necesario contrastar el cálculo con las reglas aprobadas y revisar el nombre y firma de los métodos utilizados.

Ejemplo auditado: caso CP-12

```
def test_cp12_mock_notificacion_confirma_descuento_volumen():
    notificador_mock = Mock()
    pedido = TechStoreOrder(pedido_id="P-108", cliente_id="CLI-08")
    pedido.agregar_item(sku="SKU-LAP", cantidad=2, precio_unitario=3000.0)
    total_confirmado = pedido.confirmar(notificador=notificador_mock)
    assert total_confirmado == 5700.0
    assert pedido.estado == "CONFIRMED"
    notificador_mock.enviar_confirmacion.assert_called_once_with(
        "CLI-08", "P-108", 5700.0
    )
```

Revisión manual: dos unidades de $3,000 producen un subtotal de $6,000; el descuento de 5% equivale a $300 y deja un total de $5,700. El mock permite verificar el envío de una notificación con cliente, pedido e importe exactos sin realizar conexiones externas.

3. Tres incidencias relevantes: análisis y aprendizaje

3.1 Confirmación prematura en el proceso de checkout

Problema: el servicio puede actualizar el pedido a CONFIRMED antes de conocer la decisión de la pasarela. Si el pago es rechazado, existe una inconsistencia entre el estado de negocio y el resultado financiero.

Propuesta de solución: utilizar los estados PENDING_PAYMENT, CONFIRMED y PAYMENT_FAILED; confirmar únicamente después de un cobro aceptado y contemplar compensación o rollback según el diseño transaccional.

Aprendizaje: las transiciones de estado vinculadas a servicios externos deben representar hechos ya verificados, no suposiciones.

3.2 Advertencias por conexiones SQLite sin cerrar

Observación: durante una ejecución de cobertura aparecieron mensajes ResourceWarning asociados con conexiones SQLite que quedaron abiertas. Las 27 pruebas aprobaron, pero la advertencia señala una posible necesidad de limpieza adicional.

Mejora sugerida: asegurar el cierre de sesiones y motores de prueba, y verificar la liberación de conexiones al terminar cada fixture de pytest.

Aprendizaje: una suite satisfactoria también debe cuidar la gestión de recursos para ser reproducible y evitar efectos entre pruebas.

3.3 Validación precisa del límite temporal de 24 horas

Riesgo: si una prueba depende del reloj real, las diferencias de milisegundos pueden volver ambiguo el resultado al evaluar una frontera temporal exacta.

Solución aplicada en las pruebas: inyectar valores fijos de datetime mediante el argumento now para comparar fechas deterministas.

Aprendizaje: el tiempo es una dependencia que conviene controlar durante las pruebas automatizadas.

4. Respuestas a las preguntas del reporte

4.1 ¿Por qué una prueba unitaria debe evitar bases de datos y API externas?

Porque busca verificar una regla pequeña y aislada. Una dependencia externa introduce latencia, fallas de red, datos compartidos y resultados menos repetibles. Para evaluar integración se utilizan pruebas específicas con infraestructura controlada, y para los servicios externos se emplean dobles de prueba cuando corresponde.

4.2 ¿Cuál es la diferencia entre Mock y Stub?

Un stub proporciona respuestas predefinidas que permiten probar una condición. Un mock también permite verificar interacciones, como cuántas veces se invocó un método y cuáles fueron sus argumentos. En E5, assert_called_once_with(170) comprueba el importe que recibió la pasarela simulada.

4.3 ¿Qué función cumplen commit y rollback respecto a las transacciones?

Commit confirma los cambios acumulados en la transacción; rollback revierte aquellos que todavía no se han confirmado. En combinación con las propiedades del motor y sus restricciones, estas operaciones ayudan a preservar la atomicidad y consistencia de los datos.

4.4 ¿Por qué una cobertura del 100% no garantiza que el código sea correcto?

Porque la cobertura mide qué sentencias se ejecutan, no qué tan rigurosas son las aserciones ni si todos los requisitos están bien representados. Una prueba podría recorrer una rama sin comprobar correctamente el resultado; además pueden existir riesgos de concurrencia o escenarios no contemplados.

4.5 ¿Qué ventaja aporta GitHub Actions frente a las pruebas manuales?

Ejecuta verificaciones de manera automática en un entorno distinto al de desarrollo. Esto ayuda a detectar regresiones, dependencias ausentes, problemas de formato y fallos de pruebas antes de considerar satisfactorio un cambio. No sustituye por sí solo las reglas de protección de ramas.

5. Lista de verificación final

Elemento	Evidencia de cumplimiento	Estado
Entorno y dependencias	Entorno .venv, instalación de dependencias, pytest inicial	Cumplido

Elemento	Evidencia de cumplimiento	Estado
Pruebas unitarias	E2: seis pruebas en verde, con clientes VIP y regulares	Cumplido
TDD Red–Green–Refactor	E3: fallo intencional, validación restaurada y suite final	Cumplido
Persistencia y transacciones	E4: tres pruebas, commit y rollback	Cumplido
Mocks de pago	E5: pago aprobado y rechazo VIP simulados	Cumplido
Autorización	E6: cinco pruebas y límite temporal de 24 horas	Cumplido
Ruff y cobertura ≥ 80%	E7: Ruff aprobado; cobertura global 96.04%	Cumplido
Integración continua	E8: GREEN inicial, RED por pytest y GREEN de recuperación	Cumplido
PIA: matriz y cobertura	E9: 12 casos y cobertura del módulo 95.45%	Cumplido

6. Reflexión de cierre y conclusiones

Reflexión: ¿qué defecto de negocio sería más costoso en producción?

Uno de los defectos más costosos para TechStore sería confirmar un pedido sin haber recibido la aprobación del pago. El riesgo no solo consiste en mostrar un estado equivocado: también podría provocar procesos posteriores de preparación o envío de mercancía sin un ingreso comprobado. Otro riesgo importante es reservar productos sin suficiente inventario, lo que generaría pedidos imposibles de surtir y afectaría la confianza de los clientes. Para reducir ambos escenarios, las pruebas de la pasarela de pago deben simular respuestas aprobadas y rechazadas, verificar el importe enviado al servicio y comprobar que los cambios de estado ocurren en el momento correcto. En paralelo, los casos CP-01 a CP-03 tienen que asegurar que la reserva de productos respete la cantidad disponible y rechace entradas inválidas. Estas pruebas, acompañadas por la cobertura de código y la ejecución automática del pipeline, sirven como controles preventivos. Sin embargo, una ejecución con todas las pruebas aprobadas no reemplaza el análisis de requisitos ni la revisión de nuevas condiciones de negocio. La calidad depende de mantener pruebas relevantes, comprensibles y actualizadas a medida que evoluciona la solución.

Conclusiones

La práctica permitió comprobar que las reglas de negocio pueden verificarse con pruebas pequeñas, repetibles y orientadas a resultados concretos. Las validaciones de descuentos y estados demostraron la utilidad de pytest, mientras que la integración con SQLite y SQLAlchemy permitió observar el comportamiento de commit y rollback sin utilizar información productiva.

El uso de mocks hizo posible simular respuestas de pago y notificaciones sin invocar servicios externos. La revisión de seguridad mostró que las políticas de cancelación dependen del estado, la identidad del solicitante y un límite temporal estricto.

Finalmente, Ruff y las mediciones de cobertura aportaron indicadores de calidad, y GitHub Actions demostró que un fallo de pytest puede detectarse fuera del equipo local. Los resultados de las evidencias E1–E9 constituyen una base verificable para continuar desarrollando y evaluando las reglas del PIA.